Code Explanation

SQL Generator Test Suite Explanation

This test suite is designed to validate the SQL generation functionality for sales orders data. It consists of three test cases that verify the correctness of the generated SQL queries.

Overview of the Code

The code is a Pytest test suite that tests the SQL generation functionality for sales orders data. It uses the PySpark library to create a Spark session and test the generated SQL queries. The test suite consists of three test cases: `test_generate_sales_orders_stg_sql`, `test_generate_sales_orders_sql`, and `test_sql_syntax`.

Step 1: Creating a Spark Session for Testing
The first step is to create a Spark session for testing using the `pytest.fixture` decorator. This fixture is used to create a Spark session with the necessary configuration.

```
@pytest.fixture(scope="module")
def spark():
    return SparkSession.builder
        .appName("Sales Orders SQL Generator Tests")
        .master("local[*]")
        .enableHiveSupport()
        .getOrCreate()
```

Step 2: Testing the Generation of SQL for Sales Orders Staging Table
The `test_generate_sales_orders_stg_sql` test case verifies that the SQL query generated for the sales orders staging table is correct. It checks that the generated SQL is not empty and contains the required elements and columns.

```
def test_generate_sales_orders_stg_sql():
    sql = generate_sales_orders_stg_sql()
    assert sql is not None
    assert len(sql) > 0
    assert "CREATE OR REPLACE TABLE sales_orders_comp_stg" in sql
    assert "FROM" in sql
    assert "LEFT JOIN" in sql
    assert "b_source.ord_hdr" in sql
    assert "b_source.ord_dtl" in sql
    assert "delete_flag <> 'D'" in sql
    # Check for specific columns
    assert "CompOrderNumber" in sql
    assert "CompOrderType" in sql
    assert "CompSalesOrgCompanyCode" in sql
    assert "CompLineNumber" in sql
    assert "CompShipToNumber" in sql
    assert "CompSoldToNumber" in sql
    assert "CompMaterialNumber" in sql
```

Step 3: Testing the Generation of SQL for Sales Orders Table

The `test_generate_sales_orders_sql` test case verifies that the SQL query generated for the sales orders table is correct. It checks that the generated SQL is not empty and contains the required elements, columns, and complex transformations.

```
def test_generate_sales_orders_sql():
    sql = generate_sales_orders_sql()
    assert sql is not None
    assert len(sql) > 0
    assert "CREATE OR REPLACE TABLE sales_orders_comp" in sql
    assert "FROM" in sql
    assert "LEFT JOIN" in sql
    assert "s_master.sale_orders" in sql
    assert "b_source.ord_site" in sql
    assert "delete_flag <> 'D'" in sql
    # Check for specific columns
    assert "CompSourceSystem" in sql
    assert "CompOrderNumber" in sql
    assert "CompOrderType" in sql
    assert "CompSalesOrgCompanyCode" in sql
    assert "CompLineNumber" in sql
    assert "CompSiteId" in sql
    assert "CompShipToNumber" in sql
    assert "CompSoldToNumber" in sql
    assert "CompMaterialNumber" in sql
    # Check for complex transformations
    assert "CASE" in sql
    assert "WHEN" in sql
    assert "ELSE" in sql
    assert "END" in sql
    assert "UNION" in sql
    assert "GROUP BY" in sql
```

Step 4: Testing the Syntax of the Generated SQL Queries

The `test_sql_syntax` test case verifies that the generated SQL queries are syntactically valid by attempting to parse them using the Spark SQL `EXPLAIN` statement.

```
def test_sql_syntax(spark):
    try:
        spark.sql(f"EXPLAIN {generate_sales_orders_stg_sql()}")
        spark.sql(f"EXPLAIN {generate_sales_orders_sql()}")
        assert True
    except Exception as e:
        pytest.fail(f"SQL syntax error: {str(e)}")
```